# Tensorflow II

## Advanced CNN Architectures

Fatemeh Dehrouyeh



# Contents


1. Pretrained Models
2. Advanced CNN Architecture Concepts
   - Depthwise Convolution
   - Pointwise Convolution
   - Depthwise Separable Convolution
   - Residual Connections
   - Bottleneck Blocks
   - Inverted Residual Blocks
   - Linear Bottlenecks
   - Squeeze-and-Excitation
   - Compound Scaling
   - Inception Modules
   - Factorized Convolutions

# Current Section

1 Pretrained Models

2 Advanced CNN Architecture Concepts

# Official Pretrained Model Resources

Official TensorFlow/Keras documentation and architecture references:

- **MobileNetV2**
  `TensorFlow MobileNetV2 Documentation`
- **ResNet50**
  `TensorFlow ResNet50 Documentation`
- **VGG16**
  `Keras VGG Documentation`
- **EfficientNet**
  `Keras EfficientNet Documentation`
- **InceptionV3**
  `Keras InceptionV3 Documentation`

**Complete list of pretrained Keras models:**
`https://keras.io/api/applications/`

# MobileNetV2

- Trained on ImageNet for 1000-class image classification.
- Designed for mobile and low-compute applications.
- Main idea: depthwise separable convolutions.
- Uses inverted residual blocks and linear bottlenecks.
- Architecture: initial convolution, repeated bottleneck blocks, final classifier.

**Why it is interesting**

MobileNetV2 is much smaller and faster than large CNNs, so it is useful for transfer learning when compute is limited.

Docs: MobileNetV2 Keras Docs

Paper/Figures: MobileNetV2 Paper

# ResNet50

- Trained on ImageNet for 1000-class image classification.
- Has 50 weight layers.
- Main idea: residual connections, also called skip connections.
- Uses bottleneck blocks with $1 \times 1$, $3 \times 3$, and $1 \times 1$ convolutions.
- Helps train deep networks by allowing information to skip across layers.

**Why it is interesting**

ResNet made very deep CNNs easier to train and became one of the most important CNN architectures.

Docs: ResNet50 Keras Docs

Paper/Figures: ResNet Paper

# VGG16

- Trained on ImageNet for 1000-class image classification.
- Has 16 weight layers: 13 convolutional layers and 3 fully connected layers.
- Uses a simple repeated pattern of $3 \times 3$ convolutions.
- Architecture is easy to understand, but the model has many parameters.
- Often used as a clear baseline for transfer learning.

**Why it is interesting**

VGG16 is simple and readable, but computationally heavy compared with newer models.

Docs: VGG Keras Docs

Paper/Figures: VGG Paper

# EfficientNet

- Trained on ImageNet for 1000-class image classification.
- A family of models, from EfficientNetB0 to EfficientNetB7.
- Main idea: compound scaling.
- Scales depth, width, and input resolution together.
- Uses mobile inverted bottleneck blocks and squeeze-and-excitation.

**Why it is interesting**

EfficientNet improves accuracy while keeping the model relatively efficient.

Docs: EfficientNet Keras Docs

Paper/Figures: EfficientNet Paper

# InceptionV3

- Trained on ImageNet for 1000-class image classification.
- Commonly described as a deep CNN with about 48 layers.
- Main idea: Inception modules.
- Uses multiple convolution sizes inside the same block.
- Uses factorized convolutions, such as replacing larger convolutions with smaller ones.

**Why it is interesting**

InceptionV3 improves efficiency by extracting features at different scales without making the model too expensive.

Docs: InceptionV3 Keras Docs

Paper/Figures: InceptionV3 Paper

# Current Section

1. Pretrained Models
2. Advanced CNN Architecture Concepts
   - Depthwise Convolution
   - Pointwise Convolution
   - Depthwise Separable Convolution
   - Residual Connections
   - Bottleneck Blocks
   - Inverted Residual Blocks
   - Linear Bottlenecks
   - Squeeze-and-Excitation
   - Compound Scaling
   - Inception Modules
   - Factorized Convolutions

# Current Subsection

1 Pretrained Models

2 Advanced CNN Architecture Concepts
- Depthwise Convolution
- Pointwise Convolution
- Depthwise Separable Convolution
- Residual Connections
- Bottleneck Blocks
- Inverted Residual Blocks
- Linear Bottlenecks
- Squeeze-and-Excitation
- Compound Scaling
- Inception Modules
- Factorized Convolutions

# Depthwise Convolution

In a standard convolution, each filter looks across **all input channels**.

In a **depthwise convolution**, each input channel is convolved separately. For an input with $C$ channels:

$$H \times W \times C$$

we apply a separate spatial filter to each channel.

**Key Idea**

Depthwise convolution learns spatial patterns within each channel without immediately combining information across channels.

This greatly reduces the number of parameters and computations compared with a standard convolution.

**Used in:** MobileNetV2, EfficientNet

# Why Depthwise Separable Convolution Uses Fewer Parameters

Suppose the input is an RGB image with 3 channels, and we want 64 output feature maps using $3 \times 3$ kernels.

**Standard convolution:**
Each of the 64 filters spans all 3 input channels, so each filter has $3 \times 3 \times 3 = 27$ weights.
Therefore, total weights $= 27 \times 64 = 1728$.

**Depthwise separable convolution:**
First, use one $3 \times 3$ filter for each input channel: $3 \times 3 \times 3 = 27$ weights.
Then use a $1 \times 1$ pointwise convolution to create 64 output channels: $1 \times 1 \times 3 \times 64 = 192$ weights (64 different filters, each filter has 3 weights).
Therefore, total weights $= 27 + 192 = 219$.

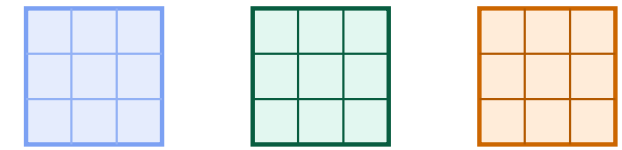
**Key Idea**

Standard convolution uses 64 full $3 \times 3 \times 3$ filters, while depthwise separable convolution uses only 3 spatial filters followed by a cheaper $1 \times 1$ convolution.

# What the $1 \times 1 \times 3 \times 64$ Means

## Step 1: Depthwise $3 \times 3$

Each channel gets its own $3 \times 3$ filter.



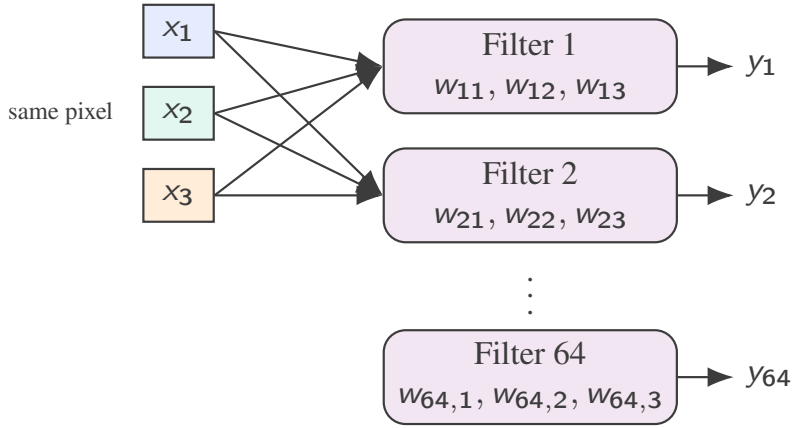

$$3 \times 3 \times 3 = 27 \text{ weights}$$

**Meaning**

Depthwise uses the **9 values** in a $3 \times 3$ patch, but stays inside **one channel**.

## Step 2: Pointwise $1 \times 1$

At the same pixel, take one value from each channel.




**Meaning**

Pointwise stays at **one pixel**, but combines the values from **all 3 channels**.

$$\underbrace{1 \times 1}_{\text{one pixel}} \times \underbrace{3}_{\text{input channels}} \times \underbrace{64}_{\text{output filters}} = 192 \text{ weights}$$

Each output filter has only three weights: $y_k = w_{k1}x_1 + w_{k2}x_2 + w_{k3}x_3$

# How the $1 \times 1$ Convolution Applies to Every Pixel

## Depthwise step

A $3 \times 3$ filter creates one value at each location.

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \quad \Longrightarrow \quad x_c(i,j)$$

**Meaning**
The $3 \times 3$ part looks around the pixel.
It handles the spatial neighborhood.

## Pointwise step

At each location, combine channels only.

$$y_k(i,j) = w_{k1}x_1(i,j) + w_{k2}x_2(i,j) + w_{k3}x_3(i,j)$$

**Meaning**
The $1 \times 1$ part stays at one pixel.
It mixes the channel values there.

## Important

The same $1 \times 1$ filter is reused at every pixel location:

$$(i,j), \quad (i,j+1), \quad (i+1,j), \quad \ldots$$

So it does **not** sum over the whole image. It sums only across channels at each pixel.

Note: The pointwise $1 \times 1$ convolution changes the number of channels from 3 to 64 while keeping the spatial size $H \times W$ the same.

# Current Subsection

1 Pretrained Models

2 Advanced CNN Architecture Concepts
- Depthwise Convolution
- Pointwise Convolution
- Depthwise Separable Convolution
- Residual Connections
- Bottleneck Blocks
- Inverted Residual Blocks
- Linear Bottlenecks
- Squeeze-and-Excitation
- Compound Scaling
- Inception Modules
- Factorized Convolutions

# Pointwise Convolution: $1 \times 1$ Convolution

A **pointwise convolution** is simply a convolution with:

$$1 \times 1$$

filters.
Although the spatial kernel is only $1 \times 1$, the filter still looks across all input channels.

It is commonly used to:

- combine information across channels,
- increase the number of channels,
- reduce the number of channels.

**Important**

A $1 \times 1$ pointwise convolution changes the number of channels, not the spatial dimensions $H \times W$.

A $1 \times 1$ convolution changes the channel representation without needing to examine neighboring pixels.
**Used in:** MobileNetV2, ResNet, EfficientNet, Inception

# Current Subsection

1. Pretrained Models
2. Advanced CNN Architecture Concepts
   - Depthwise Convolution
   - Pointwise Convolution
   - Depthwise Separable Convolution
   - Residual Connections
   - Bottleneck Blocks
   - Inverted Residual Blocks
   - Linear Bottlenecks
   - Squeeze-and-Excitation
   - Compound Scaling
   - Inception Modules
   - Factorized Convolutions

# Depthwise Separable Convolution

A **depthwise separable convolution** divides a standard convolution into two steps:

$$\text{Depthwise Convolution} \rightarrow \text{Pointwise } 1 \times 1 \text{ Convolution}$$

**Step 1:** Depthwise convolution finds spatial patterns separately in each channel.
**Step 2:** Pointwise convolution combines information across channels.

**Why Use It?**

It can perform a similar role to a standard convolution with far fewer parameters and computations.

**Major use:** MobileNet architectures

# Current Subsection

1 Pretrained Models

2 Advanced CNN Architecture Concepts
- Depthwise Convolution
- Pointwise Convolution
- Depthwise Separable Convolution
- Residual Connections
- Bottleneck Blocks
- Inverted Residual Blocks
- Linear Bottlenecks
- Squeeze-and-Excitation
- Compound Scaling
- Inception Modules
- Factorized Convolutions

# Residual or Skip Connections

Normally, a sequence of layers learns:

$$y = F(x)$$

A **residual connection** also sends the original input $x$ directly forward:

$$y = F(x) + x$$

The input therefore **skips** over several layers and is added back to their output.

**Why Is This Useful?**

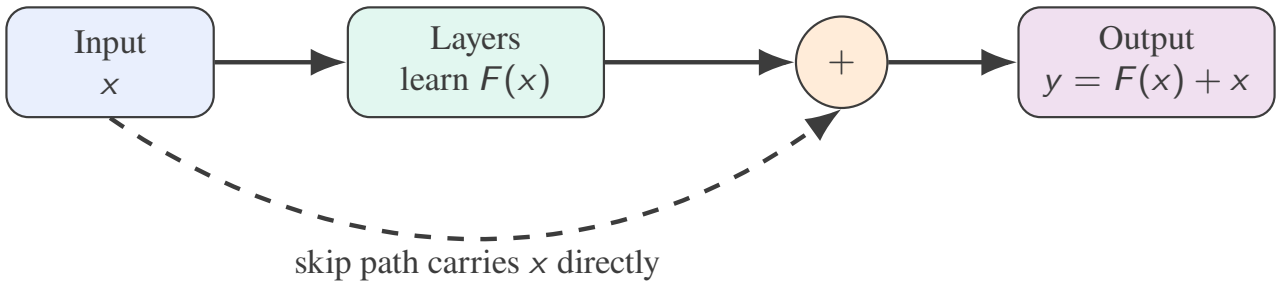
Skip connections make very deep networks easier to train because information and gradients can flow through shorter paths.

They also allow the block to focus on learning the change, or residual, relative to the original input.

**Main model:** ResNet

# Residual Connection: A Concrete Example

**The block learns a change, then adds the original input back.**




**Without skip connection**

$$y = F(x)$$

The layers must learn the full output.

**With skip connection**

$$y = F(x) + x$$

The layers only learn the change from $x$.

Input $x \rightarrow$ layers learn $F(x) \rightarrow$ add back $x$

**Key idea:** the original input gets a shorter path through the network.

# Current Subsection

1. Pretrained Models
2. Advanced CNN Architecture Concepts
   - Depthwise Convolution
   - Pointwise Convolution
   - Depthwise Separable Convolution
   - Residual Connections
   - Bottleneck Blocks
   - Inverted Residual Blocks
   - Linear Bottlenecks
   - Squeeze-and-Excitation
   - Compound Scaling
   - Inception Modules
   - Factorized Convolutions

# Bottleneck Blocks

A **bottleneck block** changes the number of channels so expensive convolutions can operate on a smaller representation.
A common ResNet bottleneck is:

$$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$$

The first $1 \times 1$ convolution reduces the channels.
The $3 \times 3$ convolution processes the smaller representation.
The final $1 \times 1$ convolution expands the channels again.

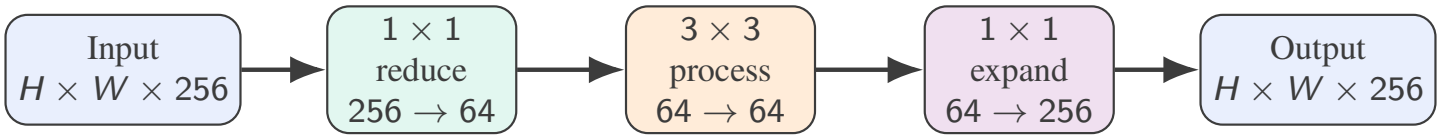
**Why "Bottleneck"?**

The middle of the block contains a narrower representation, reducing computation.

**Used in:** ResNet50 and other deep ResNet models

# Bottleneck Block: A Concrete Example

**Example: reduce 256 channels to 64, process, then expand back to 256**




**Without bottleneck**

A direct $3 \times 3$ convolution:

$$3 \times 3 \times 256 \times 256 = 589{,}824$$

weights.
(256 output filters, and each filter must look through all 256 input channels)

**With bottleneck**

$$1 \times 1 \times 256 \times 64 = 16{,}384$$

$$3 \times 3 \times 64 \times 64 = 36{,}864$$

$$1 \times 1 \times 64 \times 256 = 16{,}384$$

Total:

$$69{,}632$$

**Main idea:** the expensive $3 \times 3$ convolution runs on 64 channels, not 256.

# Current Subsection

1. Pretrained Models
2. Advanced CNN Architecture Concepts
   - Depthwise Convolution
   - Pointwise Convolution
   - Depthwise Separable Convolution
   - Residual Connections
   - Bottleneck Blocks
   - Inverted Residual Blocks
   - Linear Bottlenecks
   - Squeeze-and-Excitation
   - Compound Scaling
   - Inception Modules
   - Factorized Convolutions

# Inverted Residual Blocks

MobileNetV2 uses an **inverted residual block**.
Instead of:

$$\text{wide} \rightarrow \text{narrow} \rightarrow \text{wide}$$

as in a traditional bottleneck, MobileNetV2 often uses:

$$\text{narrow} \rightarrow \text{wide} \rightarrow \text{narrow}$$

Typical structure:

$$1 \times 1 \text{ expansion} \rightarrow \text{depthwise convolution} \rightarrow 1 \times 1 \text{ projection}$$

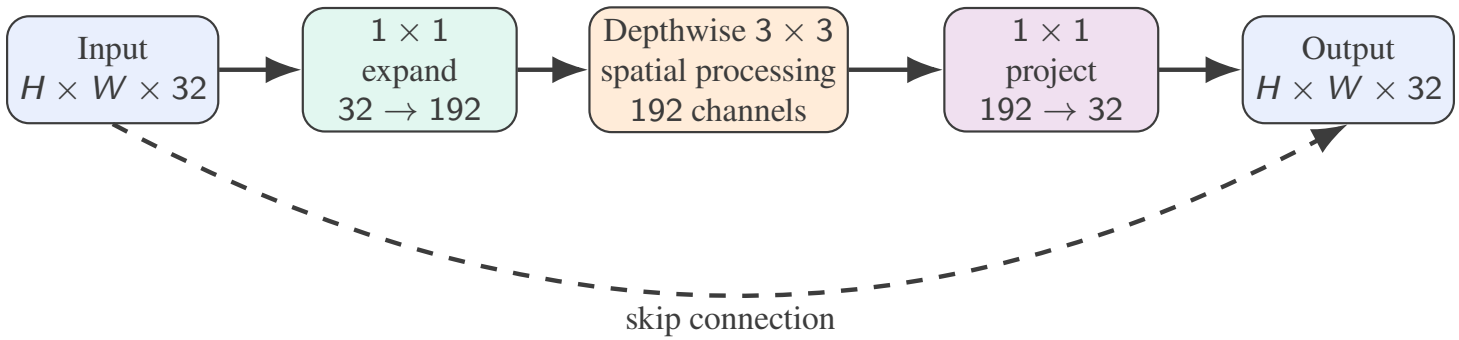
**Why "Inverted"?**

The internal representation is expanded rather than compressed, then projected back to fewer channels.

A skip connection can be added when the input and output shapes match.
**Main model:** MobileNetV2

# Inverted Residual Block: A Concrete Example

**Example: start with 32 channels, expand to 192, then project back to 32**




**Why is it "inverted"?**

A traditional bottleneck goes:

$$\text{wide} \to \text{narrow} \to \text{wide}$$

MobileNetV2 reverses this idea:

$$\boxed{32 \to 192 \to 32}$$

The middle representation is **wider**, not narrower.

**What happens inside?**

$1 \times 1$ expansion:

$$32 \to 192$$

Depthwise $3 \times 3$:

192 channels processed separately

$1 \times 1$ projection:

$$192 \to 32$$

# Continued

**Key idea:**

$$\text{Narrow} \xrightarrow{1\times1} \text{Wide} \xrightarrow{3\times3\ \text{depthwise}} \text{Wide} \xrightarrow{1\times1} \text{Narrow}$$

If the input and output shapes match, the original input can be added through a **skip connection**.

# Current Subsection

1. Pretrained Models
2. Advanced CNN Architecture Concepts
   - Depthwise Convolution
   - Pointwise Convolution
   - Depthwise Separable Convolution
   - Residual Connections
   - Bottleneck Blocks
   - Inverted Residual Blocks
   - Linear Bottlenecks
   - Squeeze-and-Excitation
   - Compound Scaling
   - Inception Modules
   - Factorized Convolutions

# Linear Bottlenecks

MobileNetV2 uses a **linear bottleneck** at the end of its inverted residual block.
Normally, convolutional layers are often followed by nonlinear activations such as ReLU.
However, the final narrow projection in MobileNetV2 does **not** use ReLU.

$$\text{Expanded Features} \rightarrow \text{Linear Projection} \rightarrow \text{Bottleneck}$$

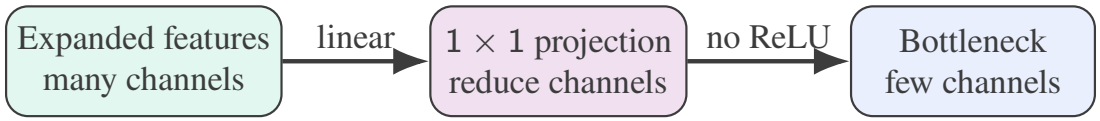
**Key Idea**

Applying a strong nonlinearity to a low-dimensional representation can destroy useful information. The linear projection helps preserve that information.

**Main model:** MobileNetV2

# Linear Bottleneck: Why No ReLU at the End?

**MobileNetV2 keeps the final narrow projection linear.**




**With ReLU**

negative values $\rightarrow 0$

In a narrow space, this can remove useful information.

**With linear projection**

values stay unchanged

The bottleneck keeps more information.

Expanded features $\rightarrow$ Linear projection $\rightarrow$ Bottleneck

**Key idea:** do not use ReLU after the final narrow projection, because it can destroy information in a low-dimensional representation.

# Current Subsection

1. Pretrained Models
2. Advanced CNN Architecture Concepts
   - Depthwise Convolution
   - Pointwise Convolution
   - Depthwise Separable Convolution
   - Residual Connections
   - Bottleneck Blocks
   - Inverted Residual Blocks
   - Linear Bottlenecks
   - Squeeze-and-Excitation
   - Compound Scaling
   - Inception Modules
   - Factorized Convolutions

# Squeeze-and-Excitation

A **Squeeze-and-Excitation (SE) block** allows the network to learn which feature channels are most important.
It works in two main steps:

$$\text{Squeeze} \rightarrow \text{Excitation}$$

**Squeeze:**
Global Average Pooling summarizes each feature map into one value.

$$H \times W \times C \rightarrow C$$

**Excitation:**
The network learns a weight for each channel and uses those weights to rescale the feature maps.

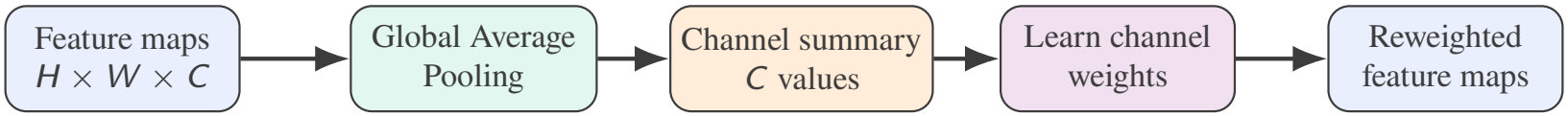
**Key Idea**

The network learns to emphasize useful channels and suppress less useful channels.

**Used in:** EfficientNet

# Squeeze-and-Excitation: A Concrete Example

**Goal: learn which feature channels are important and reweight them.**




**Step 1: Squeeze**

Each feature map is summarized into one number:

$$H \times W \times C \to C$$

So if there are 64 channels:

$$H \times W \times 64 \to 64 \text{ values}$$

**Step 2: Excitation**

The network learns one importance weight per channel:

$$s_1, s_2, \ldots, s_C$$

Then each channel is rescaled:

$$\text{Channel}_c \to s_c \cdot \text{Channel}_c$$

Feature maps $\to$ Squeeze $\to$ Channel weights $\to$ Rescale channels

**Key idea:** important channels get larger weights, while less useful channels get smaller weights.

# Current Subsection

1. Pretrained Models
2. Advanced CNN Architecture Concepts
   - Depthwise Convolution
   - Pointwise Convolution
   - Depthwise Separable Convolution
   - Residual Connections
   - Bottleneck Blocks
   - Inverted Residual Blocks
   - Linear Bottlenecks
   - Squeeze-and-Excitation
   - Compound Scaling
   - Inception Modules
   - Factorized Convolutions

# Compound Scaling

CNNs can be made larger in three main ways:

- **Depth:** add more layers,
- **Width:** use more channels,
- **Resolution:** use larger input images.

Earlier approaches often increased only one of these.

**EfficientNet** scales all three together:

$$\text{Depth} + \text{Width} + \text{Resolution}$$

**Key Idea**

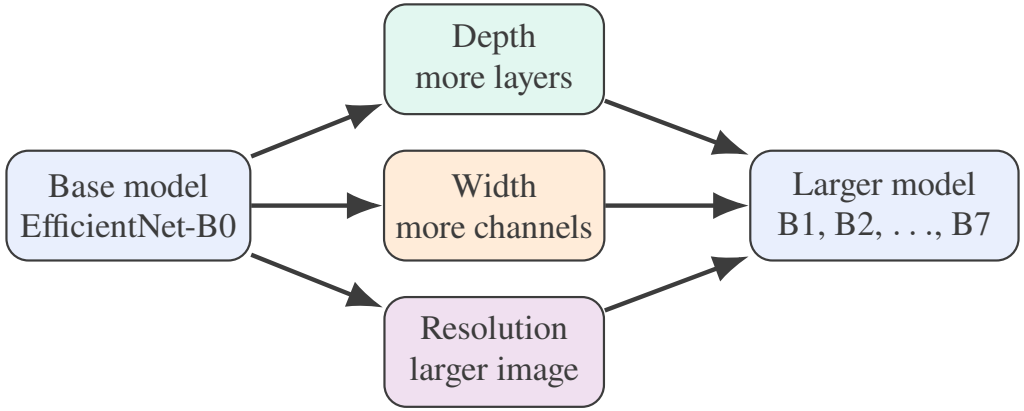
Compound scaling increases model capacity in a balanced way instead of arbitrarily making only one dimension larger.

This idea produces the EfficientNet family:

$$B0,\ B1,\ B2, \ldots, B7$$

# Compound Scaling: Balanced Growth

**EfficientNet scales three things together instead of only one.**




**Depth**
More layers.

$L \uparrow$

**Width**
More channels.

$C \uparrow$

**Resolution**
Larger input.

$H, W \uparrow$

$$\text{Compound scaling} = \text{Depth} + \text{Width} + \text{Resolution}$$

**Key idea:** grow the network in a balanced way, instead of making only one part bigger.

# Current Subsection

1. Pretrained Models
2. Advanced CNN Architecture Concepts
   - Depthwise Convolution
   - Pointwise Convolution
   - Depthwise Separable Convolution
   - Residual Connections
   - Bottleneck Blocks
   - Inverted Residual Blocks
   - Linear Bottlenecks
   - Squeeze-and-Excitation
   - Compound Scaling
   - Inception Modules
   - Factorized Convolutions

# Inception Modules

An **Inception module** processes the same input through several branches in parallel.
For example:

$1 \times 1$ Conv
$3 \times 3$ Conv
$5 \times 5$ Conv
Pooling

The outputs of the branches are then concatenated.

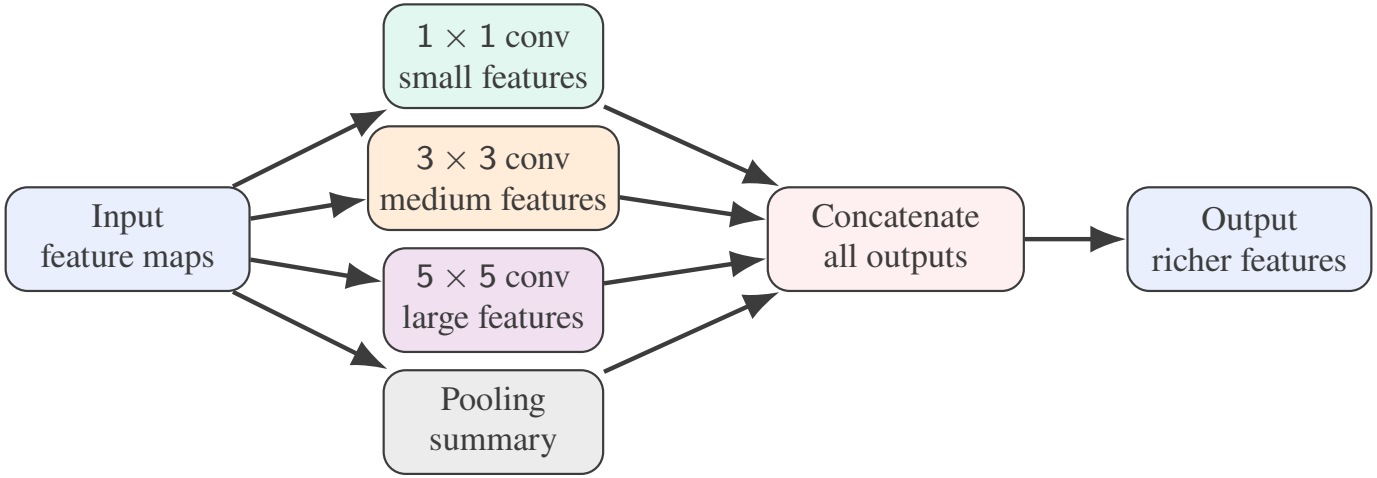
**Why Is This Useful?**

Different kernel sizes can detect visual patterns at different spatial scales.

Instead of choosing only one kernel size, the network extracts several types of features at the same stage.
**Main model:** Inception

# Inception Module: Parallel Branches

**The same input is processed by several operations in parallel.**




**Why use branches?**
Different kernel sizes detect different spatial patterns. $1 \times 1$, $3 \times 3$, $5 \times 5$

Small kernels capture local details. Larger kernels capture wider context.

**Why concatenate?**
Each branch produces useful features. Concatenation stacks them along the channel dimension:

$$C_1 + C_2 + C_3 + C_4$$

So the next layer receives all feature types.

same input $\rightarrow$ parallel filters $\rightarrow$ concatenate outputs

# Current Subsection

1. Pretrained Models
2. Advanced CNN Architecture Concepts
   - Depthwise Convolution
   - Pointwise Convolution
   - Depthwise Separable Convolution
   - Residual Connections
   - Bottleneck Blocks
   - Inverted Residual Blocks
   - Linear Bottlenecks
   - Squeeze-and-Excitation
   - Compound Scaling
   - Inception Modules
   - Factorized Convolutions

# Factorized Convolutions

A large convolution can sometimes be replaced by several smaller convolutions. For example:

$$5 \times 5$$

can be approximated using:

$$3 \times 3 \rightarrow 3 \times 3$$

In InceptionV3, convolutions can also be factorized asymmetrically:

$$3 \times 3 \rightarrow 1 \times 3 \rightarrow 3 \times 1$$

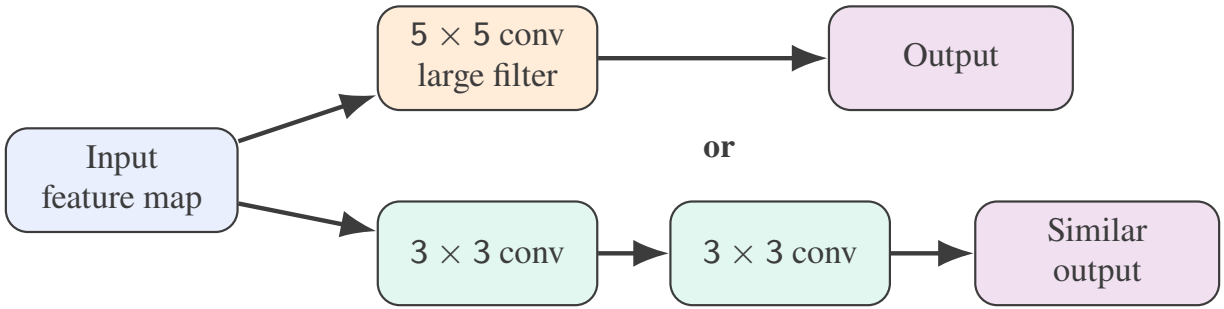
**Why Factorize?**

Smaller convolutions can reduce computation and parameters while still producing a large effective receptive field.

**Main model:** InceptionV3

# Factorized Convolutions: Replace Big Filters

**Large convolutions can be replaced by smaller convolutions.**




**Example 1**
Instead of one large filter:

$$5 \times 5$$

use two smaller filters:

$$3 \times 3 \rightarrow 3 \times 3$$

This keeps a large effective receptive field.

**Example 2**
In InceptionV3, a square filter can be split:

$$3 \times 3 \rightarrow 1 \times 3 \rightarrow 3 \times 1$$

This reduces parameters and computation.

large filter $\rightarrow$ smaller filters $\rightarrow$ similar receptive field with less cost

# Which Architecture Uses Which Idea?

| Architecture | Important Ideas |
|---|---|
| VGG16 | Repeated $3 \times 3$ convolutions |
| ResNet50 | Residual connections, bottleneck blocks, $1 \times 1$ convolutions |
| MobileNetV2 | Depthwise separable convolutions, inverted residuals, linear bottlenecks |
| EfficientNet | MBConv blocks, squeeze-and-excitation, compound scaling |
| InceptionV3 | Inception modules, parallel branches, factorized convolutions |

**Big Picture**

These models use the same basic CNN components we already learned, but arrange and modify them in more efficient ways.

# Congratulations!

Now you know about advanced CNN architectures!

